czii-cryo-et-object-identification

1位解法 [物体検出パート] ＋学習コード＋推論＋モデル公開

Rank	1
Team	Daddies
Author	Eugene Khvedchenya
Topic ID	561440
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/czii-cryo-et-object-identification/discussion/561440
Retrieved with Kaggle CLI on 2026-10-21.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

```
Throughout the competition there were numerous missile strikes, bombings, and other acts of war that
have taken the lives of many innocent people in Ukraine.
Rockets from russia hit within few kilometers from my home in Odesa. Each day Kaggle users from
Ukraine facing the chance of not waking up. Just keep in this mind while you read this solution
writeup.

I would like to thank the Armed Forces of Ukraine, the Security Service of Ukraine, Defence
Intelligence of Ukraine, and the State Emergency Service of Ukraine for providing safety and security
to participate in this great competition, complete this work, and help science, technology, and
business not to stop but to move forward.
```

（訳）コンペ期間中、ウクライナでは多数のミサイル攻撃や爆撃などの戦争行為があり、多くの罪のない人々の命が奪われました。ロシアのロケットは、オデーサにある私の自宅から数キロ以内に着弾しました。ウクライナのKaggleユーザーは、毎日、翌朝目覚められないかもしれないという状況に直面しています。この解法記事を読む際には、どうかこのことを心に留めておいてください。

このすばらしいコンペに参加し、この仕事をやり遂げ、科学・技術・ビジネスが止まることなく前進し続けるための安全と安心を提供してくれた、ウクライナ軍、ウクライナ保安庁、ウクライナ国防情報総局、ウクライナ国家非常事態庁に感謝します。

TLDR: 解法は、セグメンテーションモデル（ResNet と B3 エンコーダを持つ 3D Unet）と、Monai の物体検出モデル（SegResNet と DynUnet backbone）のアンサンブルです。

NVidia TensorRT で推論を高速化し、eager な PyTorch 実行と比べて 200% の高速化を達成しました。また、2枚の T4 GPU で予測を並列に実行しています。

チームメイトの @christofhenkel には、私たちのモデルを最終提出用にアンサンブルしてくれたすばらしい仕事に感謝します。それぞれ単独では Top-5 に入ることができましたが、統合した途端に Top-1 へ跳ね上がりました。

この記事では、解法のうち私のパートを説明します。セグメンテーションのアプローチとアンサンブル手法については、Christof の記事を参照してください。

更新: 物体検出（OD）パートの学習コード、モデル、推論コードを公開しました。こちらを確認してください。

- 学習コード https://github.com/BloodAxe/Kaggle-2024-CryoET
- 学習済みモデルのデータセット https://www.kaggle.com/datasets/bloodaxe/cryoet-detection-models（GitHub のミラー: https://github.com/BloodAxe/Kaggle-2024-CryoET/releases/tag/1.0）
- [Notebook] ONNX から TensorRT エンジンへの変換 https://www.kaggle.com/code/bloodaxe/convert-onnx-ensemble-to-tensorrt
- [Notebook] 推論 notebook（OD のみ） https://www.kaggle.com/code/bloodaxe/v4-inference-notebook/notebook
- [Dataset] ONNX アンサンブル https://www.kaggle.com/datasets/bloodaxe/cryoet-onnx-models/

はじめに

私のベースラインは、SegResNet backbone で構築したセグメンテーション（ヒートマップ）3D Unet でした。教師ありの目的変数として、ボリューム内に存在する正解ターゲットの位置にガウス分布のピークを置いた、5クラ

スのヒートマップを作りました。モデルの学習には CenterNet 論文の reduced focal loss を使い、同じ論文の NMS 後処理のアプローチを使いました。

このアプローチはすぐに LB で約 0.740（提出時点で銀圏）を出しましたが、多くの参加者がセグメンテーションのアプローチを使うと予想したので、私はこれを捨てました。セグメンテーションは最も実装しやすい一方で、点検出タスクに最適とは限りません（モデルの容量の多くが、粒子の識別ではなく、ピークにおけるガウス分布の曲率の予測に使われてしまいます）。2つ目の理由はモデルの多様性です。将来チームを組むことになった場合、最終的なアンサンブルの多様性が少なくなってしまいます。

モデリングのアプローチ

最終的なアプローチでは、Monai の SegResNet と DynUnet の backbone に、独自の点検出 head を組み合わせました。YOLO ベースの物体検出モデルを作ってきた経験から、anchor-free の点検出モデルを実装しました。box 検出と同様に、モデルはクラス確率マップと、物体中心へのオフセットを予測します。ただし box 検出と違って bounding box という概念はないので、2点間の IoU として、よく知られた object keypoint similarity の指標 exp(-mse(x,y)/ (2 * radius ^ 2)) を IoU 指標の代わりに使いました。

このモデルを学習するために、PP-Yolo の loss 関数をいくつか変更して模倣した独自の loss 関数を実装しました。

- 点と点の IoU に基づいて、正解ラベルを予測中心に割り当てる。Point-Point IoU 指標。
- 各正解ラベルについて Top-K の予測を学習に使う
- クラスマップの予測には varifocal loss を、距離の回帰には IoU ベースの loss を計算する

私の点検出のアプローチは、精度と学習速度の両面でうまく機能しました。4x3090 で、1 fold の学習はわずか2時間でした。単一 fold の検出モデルの最初の提出は、LB で 0.752 でした。モデリングの方針が固まった後は、それを高速化することに力を注ぎました。

このコンペでは、500 本のスキャンを 12 時間の制限内で処理する必要があります。これは1スタディあたり約 1m30s に相当します。私のモデルでは、クラスマップ [B,C,D/2,H/2,W/2] とオフセットマップ [B,3,D/2,H/2,W/2] を、深さ・高さ・幅のそれぞれで stride 2 で予測します。

物体検出のアプローチでは、フル解像度の特徴マップを予測する必要はありません。小さな特徴マップを予測すると、モデルのスループットに大きく効きます。stride 1（フル解像度）から stride 2 の出力に変えると、ほぼ 50% 速くなりました。

stride 1、stride 2、stride 4、stride 2 & 4 の出力で ablation を行い、次のことが分かりました。

- stride 1 は、stride 2 よりわずかに、ほんのわずかに良い（0.002）LB スコア
- stride 4 もそれなりに機能したが、F-beta スコアでは stride 2 の方が良かった
- stride 2 と 4 を両方使っても、stride 2 だけの場合より改善しなかった。2つの head を使う方法で興味深い観察があり、大きな粒子は stride 4 に移り、小さなクラスは stride 2 のクラスマップに現れる傾向があった。

すべてのモデルで stride 2 の予測マップを使っています。

学習

すべてのモデルで 5-fold の交差検証を使いました。各 fold では、2 スタディを検証に、5 スタディを学習に使いました。

学習の1エポックでは、スタディごとに一定数のランダムなクロップと、各粒子インスタンスの周りの一定数のランダムなクロップを使いました。データ拡張には、Z 軸周りのランダムな回転（±180度）、X 軸と Y 軸周りのわずかな回転（±10）、ごく小さなスケールのゆらぎ（±5%）、X、Y、Z 軸に沿ったランダムな反転を使いました。

入力ボリュームから粒子を消す、あるスタディから別のスタディへ粒子をランダムにコピー＆ペーストする、粒子インスタンスに mixup をかける、といった実験もしましたが、これらのデータ拡張手法は最終モデルの改善にはつながりませんでした。

検証エポックでは、推論カーネルと同じウィンドウサイズとオーバーラップの sliding window を使いました。検証中は、個々のタイルを最終的なクラスマップとオフセットマップに集約し、その最終マップで F-beta スコアを計算しました。各エポックの後、検証セットで F-beta スコアを最大化するクラスごとの閾値を計算しました。学習実験ごとに上位5つのモデルを保存し、後でそれらを平均しました。これはほぼ常に F-beta スコアを上げました。

学習には 96x128x128px を使い、検証は 192x128x128 のボリュームで行いました。

タイリング

入力ボリュームのタイリングには sliding window を使いました。ウィンドウサイズは 192x128x128px で、184x630x630 の入力ボリュームに対して 1x9x9 のタイル構成です。個々のタイルは、タイルの境界にあるピクセルほど重みが小さく、中心ほど大きくなる重み付き平均で、最終的なクラスマップとオフセットマップに集約しました。sliding average による集約は自前で実装しました。

後処理

形状 [C, D, H, W] の集約済みクラスマップと、形状 [3, D, H, W] のオフセットマップに対して、次の後処理を行いました。

- 重複検出を取り除く CenterNet 風の NMS: x = x * (x == maxpool3d(x, kernel_size=3, stride=1, padding=1))
- 各クラスについて Top-K の予測を取る（私は 16K を使いましたが、もっと小さい数でもうまくいくと思います）
- 確信度の閾値で予測をフィルタする（各クラスに固有の確信度閾値がある）
- 1つのクラスの検出に対して、ペアごとの IoU 距離を使った greedy NMS を適用する（Box NMS の仕組みと同様）
- ピクセルをオングストロームにスケールする

私のモデルは粒子の中心へのオフセットを直接予測しているので、このアプローチは点の座標の系統的なずれに影響されません（0.5px だと言う人もいれば、1px だと主張する人もいます）。

物体検出のアンサンブル

最終的な物体検出のアンサンブルは、OD SegResNet の5モデル（fold）と OD DynUnet の5モデルです（OD は Object Detection の略で、セグメンテーションベースのモデルと区別するために使っています）。

合計 10 モデルです。アンサンブルに入れる最終チェックポイントの選択には、ローカルの F-beta 指標を使いました。

SegResNet

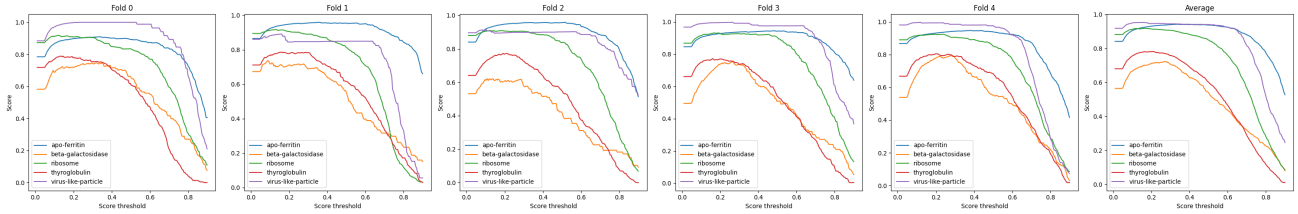
fold	score	AFRT	BGT	RBSM	TRGLB	VLP
0	0.8457	0.265	0.29	0.195	0.15	0.55
1	0.8366	0.345	0.11	0.185	0.135	0.305
2	0.8046	0.355	0.28	0.395	0.34	0.185
3	0.8398	0.23	0.345	0.405	0.36	0.255
4	0.8437	0.165	0.35	0.245	0.235	0.27

DynUnet

fold	score	AFRT	BGT	RBSM	TRGLB	VLP
0	0.8272	0.145	0.33	0.215	0.235	0.405
1	0.8337	0.39	0.185	0.215	0.21	0.235
2	0.7971	0.425	0.16	0.55	0.275	0.65
3	0.8418	0.385	0.345	0.345	0.305	0.21
4	0.842	0.23	0.34	0.255	0.375	0.14

個々の fold の F-4 スコアを見ると分かるように、CV と LB のスコアの間には大きな差があります。この現象についての私の説明は、ローカルで使う検証セットと LB とでスタディの数が大きく違うことです。反転や90度回転したスタディで検証セットを拡張すると、CV スコアは下がり、F-beta 曲線もより滑らかになることが分かりました。これにより、より正確でノイズの少ない閾値を見つけられます。

最終アンサンブルの閾値を決めるために、クラスごとに OOF の F-beta 曲線を計算し、平均 F-beta スコアを最大化する閾値を選びました。



推論の高速化

最終提出では、推論に TensorRT を使っています。

最初は torch.jit を使っていて、始めのうちはそれで十分でしたが、@christofhenkel とチームを組んでからは、より効率的なアプローチが必要になりました。

CUDAExecutionProvider を使った onnxruntime も試しましたが、torch.jit とほぼ同じスループットでした。

最終的に、TensorRT の性能をフルに引き出す TensorRTExecutionProvider を有効にすることで、torch.jit と比べて目標としていた 200% の高速化を達成できました。

個々のチェックポイントから TensorRT エンジンまでの流れは、複数の段階からなります。

- [オフライン] 個々のチェックポイントを、全モデルを含みそれらの予測を平均する1つの ONNX モデルに変換する
- [Kaggle] ONNX モデルを TensorRT エンジンに変換する（そしてディスクに保存する）。この手順は Kaggle 上で、推論に使うターゲットの GPU である T4 を使って行います。これにより、提出時に TensorRT エンジンを作る 10 分を節約できました。
- [Kaggle] 実際の推論 notebook。テストデータ全体を2つのチャンクに分け、2xT4 GPU で並列に処理します。その後、2つの提出シャードを連結して最終提出を作ります。

私のアンサンブルの結果を @christofhenkel のアンサンブルと組み合わせるために、クラスマップのレベルで予測の重み付き平均をとり、その後に上記の後処理を行いました。解法の2つ目のパートはこちらを確認してください。

（私にとっては）あまりうまくいかなかったこと

- Mixup、Copy-Paste、Random-Erasing（CV は上がり、LB は下がった）
- 2.5-D モデル
- HRNet と ConvNext の 3D 版
- ガウスノイズ、異方的なスケールのゆらぎ
- 知識蒸留

参考

- 学習コード https://github.com/BloodAxe/Kaggle-2024-CryoET
- 学習済みモデルのデータセット https://www.kaggle.com/datasets/bloodaxe/cryoet-detection-models（GitHub のミラー: https://github.com/BloodAxe/Kaggle-2024-CryoET/releases/tag/1.0）
- [Notebook] ONNX から TensorRT エンジンへの変換 https://www.kaggle.com/code/bloodaxe/convert-onnx-ensemble-to-tensorrt
- [Notebook] 推論 notebook（OD のみ） https://www.kaggle.com/code/bloodaxe/v4-inference-notebook/notebook
- [Dataset] ONNX アンサンブル https://www.kaggle.com/datasets/bloodaxe/cryoet-onnx-models/

補足Q&A

Yassine Alouini — 質問

記事をありがとうございます。

物体検出パートでは MixUp が効かなかったとのことですが、セグメンテーションパートでは効いたようです。

なぜだと思いますか。

Eugene Khvedchenya — 回答

良い質問ですね。正直なところ、分かりません。学習データに少なくとも 100 スタディあれば、CV の推定はもっとノイズが少なくなり、何が効いて何が効かないかについて、より信頼できるシグナルが得られたのではないかと思います。

この指標は選ぶ確信度閾値にとても敏感なので、LB の public 部分のスタディ数を考えると、閾値を調整する現実的な方法は LB での検証しかありませんでした。public に過学習させるのは好きではありませんが、このコンペではそれが唯一の現実的な選択肢でした。このような条件では、あるアプローチが効く・効かないと自信を持って言うのは本当に難しいです。単に閾値の悪い領域にいるだけかもしれず、ある閾値を +0.05、別の閾値を -0.05 動かすだけで SOTA になるかもしれないからです。mixup のモデルについても閾値を少しいじってみましたが、最良の組み合わせを探すのに使った提出は5回以下でした。